

Project 04: E-Commerce Backend

Difficulty: Intermediate | Estimated Time: 2 Weeks

1. Project Overview and Goals

This project builds the relational database backend for a full-featured e-commerce platform. The schema supports product catalogs with variants, a shopping cart, order lifecycle management, payments, shipping, customer reviews, and promotions. It mirrors the complexity of real production e-commerce databases used at companies like Shopify, Amazon, or Etsy.

Goals:

- Model a product catalog with variants (color, size, SKU-level pricing).
 - Implement a transactional order workflow with state machine status transitions.
 - Handle payments, refunds, and split payments.
 - Design a promotions/discount engine with rule-based logic.
 - Build analytics queries for business intelligence (conversion, revenue, cohort).
-

2. Learning Objectives

- Model product variants using a flexible attribute-value pattern.
 - Implement optimistic locking for cart/order concurrency.
 - Use JSONB for flexible product attributes and shipping metadata.
 - Write window functions for revenue cohort analysis.
 - Practice `LATERAL` joins for "top N per group" queries.
 - Implement a state machine in SQL using `CHECK` and trigger constraints.
 - Use `EXCLUDE` constraints for non-overlapping promotions.
 - Master multi-step transaction logic for order placement.
-

3. Functional Requirements

- **Products:** Title, description, images, variants, pricing tiers, tags.
 - **Customers:** Registration, address book, account balance.
 - **Cart:** Persistent cart with expiry, item quantity management.
 - **Orders:** Order placement, line items, shipping info, status lifecycle.
 - **Payments:** Multiple payment methods per order, refunds.
 - **Promotions:** Discount codes, percentage/fixed/BOGO discounts.
 - **Reviews:** Customer product reviews with ratings and moderation.
 - **Analytics:** Revenue, top products, customer lifetime value.
-

4. Non-Functional Requirements

- Prices stored in cents (`INTEGER`) to avoid floating-point errors.
 - All state transitions validated (e.g., can't go from Shipped to Pending).
 - Optimistic locking on order updates using `version` column.
 - JSONB for product metadata and shipping address snapshots.
 - Indexes on all foreign keys and common filter columns.
-

5. Complete Database Schema

```
-- =====
-- E-COMMERCE BACKEND - Complete Schema
-- PostgreSQL 15+
-- =====

CREATE SCHEMA IF NOT EXISTS shop;
SET search_path = shop, public;

-- Prices in cents
CREATE DOMAIN price_cents AS INTEGER CHECK (VALUE >= 0);
CREATE DOMAIN pct_discount AS NUMERIC(5,2) CHECK (VALUE BETWEEN 0 AND 100);

-- -----
-- CUSTOMERS
-- -----

CREATE TABLE customers (
    customer_id    SERIAL PRIMARY KEY,
    email          VARCHAR(300) NOT NULL UNIQUE,
    first_name     VARCHAR(100) NOT NULL,
    last_name      VARCHAR(100) NOT NULL,
    phone          VARCHAR(30),
    password_hash  VARCHAR(256),
    is_verified    BOOLEAN NOT NULL DEFAULT FALSE,
    is_active      BOOLEAN NOT NULL DEFAULT TRUE,
    account_credit price_cents NOT NULL DEFAULT 0,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_login_at  TIMESTAMPTZ
);

CREATE INDEX idx_customers_email  ON customers (email);
CREATE INDEX idx_customers_active ON customers (is_active) WHERE is_active = TRUE;

CREATE TABLE customer_addresses (
    address_id    SERIAL PRIMARY KEY,
    customer_id   INTEGER NOT NULL REFERENCES customers(customer_id) ON DELETE
CASCADE,
    label        VARCHAR(50) DEFAULT 'Home',
    full_name     VARCHAR(200) NOT NULL,
    line1        VARCHAR(300) NOT NULL,
    line2        VARCHAR(300),
    city         VARCHAR(100) NOT NULL,
    state        VARCHAR(100),
    postal_code   VARCHAR(20) NOT NULL,
    country      VARCHAR(100) NOT NULL DEFAULT 'US',
    phone        VARCHAR(30),
    is_default    BOOLEAN NOT NULL DEFAULT FALSE
);
```

```

-- Ensure only one default address per customer
CREATE UNIQUE INDEX idx_addr_default ON customer_addresses (customer_id)
    WHERE is_default = TRUE;

-----

-- CATEGORIES
-----

CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    slug        VARCHAR(100) NOT NULL UNIQUE,
    name        VARCHAR(100) NOT NULL,
    parent_id   INTEGER REFERENCES categories(category_id),
    sort_order  SMALLINT DEFAULT 0
);

-----

-- PRODUCTS
-----

CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    slug        VARCHAR(300) NOT NULL UNIQUE,
    title       VARCHAR(400) NOT NULL,
    description  TEXT,
    category_id INTEGER REFERENCES categories(category_id),
    brand       VARCHAR(200),
    tags        TEXT[],
    metadata    JSONB NOT NULL DEFAULT '{}',
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    is_featured BOOLEAN NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_products_category ON products (category_id);
CREATE INDEX idx_products_active   ON products (is_active) WHERE is_active = TRUE;
CREATE INDEX idx_products_tags     ON products USING GIN (tags);
CREATE INDEX idx_products_metadata ON products USING GIN (metadata);
CREATE INDEX idx_products_fts      ON products USING GIN (
    to_tsvector('english', title || ' ' || COALESCE(description, ''))
);

-----

-- PRODUCT VARIANTS
-----

CREATE TABLE product_variants (
    variant_id SERIAL PRIMARY KEY,
    product_id INTEGER NOT NULL REFERENCES products(product_id) ON DELETE
CASCADE,
    sku        VARCHAR(100) NOT NULL UNIQUE,
    title      VARCHAR(200) NOT NULL DEFAULT 'Default',
    price_cents price_cents NOT NULL,
    compare_price price_cents, -- original price for "was $X" display

```

```

cost_cents      price_cents,
weight_grams    INTEGER CHECK (weight_grams >= 0),
inventory_qty    INTEGER NOT NULL DEFAULT 0 CHECK (inventory_qty >= 0),
inventory_policy VARCHAR(20) NOT NULL DEFAULT 'deny'
                CHECK (inventory_policy IN ('deny','continue')),
attributes       JSONB NOT NULL DEFAULT '{}', -- {"color":"Red","size":"L"}
is_active        BOOLEAN NOT NULL DEFAULT TRUE,
created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_variants_product ON product_variants (product_id);
CREATE INDEX idx_variants_sku     ON product_variants (sku);
CREATE INDEX idx_variants_attrs   ON product_variants USING GIN (attributes);
CREATE INDEX idx_variants_active  ON product_variants (product_id) WHERE is_active
= TRUE;

-- -----
-- PRODUCT IMAGES
-- -----

CREATE TABLE product_images (
    image_id      SERIAL PRIMARY KEY,
    product_id    INTEGER NOT NULL REFERENCES products(product_id) ON DELETE CASCADE,
    variant_id    INTEGER REFERENCES product_variants(variant_id) ON DELETE SET NULL,
    url           VARCHAR(500) NOT NULL,
    alt_text      VARCHAR(300),
    sort_order    SMALLINT DEFAULT 0,
    is_primary    BOOLEAN NOT NULL DEFAULT FALSE
);

-- -----
-- PROMOTIONS / DISCOUNT CODES
-- -----

CREATE TABLE promotions (
    promo_id      SERIAL PRIMARY KEY,
    code          VARCHAR(50) NOT NULL UNIQUE,
    description    TEXT,
    discount_type  VARCHAR(20) NOT NULL
                CHECK (discount_type IN
('percentage','fixed_amount','free_shipping','bogo')),
    discount_value NUMERIC(10,2) NOT NULL CHECK (discount_value > 0),
    min_order_cents price_cents DEFAULT 0,
    max_uses        INTEGER,
    uses_per_customer SMALLINT DEFAULT 1,
    used_count      INTEGER NOT NULL DEFAULT 0,
    starts_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at      TIMESTAMPTZ,
    is_active        BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_promo_code ON promotions (code) WHERE is_active = TRUE;

-- -----

```

```

-- CARTS
-----
CREATE TABLE carts (
  cart_id      UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  customer_id  INTEGER REFERENCES customers(customer_id) ON DELETE SET NULL,
  session_id   VARCHAR(100),
  expires_at   TIMESTAMPTZ NOT NULL DEFAULT (NOW() + INTERVAL '30 days'),
  promo_code   VARCHAR(50),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE cart_items (
  item_id      SERIAL PRIMARY KEY,
  cart_id      UUID NOT NULL REFERENCES carts(cart_id) ON DELETE CASCADE,
  variant_id   INTEGER NOT NULL REFERENCES product_variants(variant_id),
  quantity     SMALLINT NOT NULL DEFAULT 1 CHECK (quantity > 0),
  added_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (cart_id, variant_id)
);

CREATE INDEX idx_cart_customer ON carts (customer_id);
CREATE INDEX idx_cart_items   ON cart_items (cart_id);

-----
-- ORDERS
-----
CREATE TABLE orders (
  order_id      SERIAL PRIMARY KEY,
  order_number   VARCHAR(30) NOT NULL UNIQUE,
  customer_id   INTEGER REFERENCES customers(customer_id),
  email         VARCHAR(300) NOT NULL,
  status        VARCHAR(30) NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'confirmed', 'processing',
'shipped', 'delivered', 'cancelled', 'refunded')),
  subtotal_cents price_cents NOT NULL DEFAULT 0,
  discount_cents price_cents NOT NULL DEFAULT 0,
  shipping_cents price_cents NOT NULL DEFAULT 0,
  tax_cents      price_cents NOT NULL DEFAULT 0,
  total_cents    price_cents NOT NULL DEFAULT 0,
  currency       VARCHAR(3) NOT NULL DEFAULT 'USD',
  promo_id       INTEGER REFERENCES promotions(promo_id),
  shipping_address JSONB NOT NULL DEFAULT '{}',
  billing_address JSONB NOT NULL DEFAULT '{}',
  notes          TEXT,
  version        INTEGER NOT NULL DEFAULT 1,
  placed_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_orders_customer ON orders (customer_id);

```

```

CREATE INDEX idx_orders_status    ON orders (status);
CREATE INDEX idx_orders_placed    ON orders (placed_at DESC);
CREATE INDEX idx_orders_number    ON orders (order_number);

-----
-- ORDER LINE ITEMS
-----

CREATE TABLE order_items (
    item_id        SERIAL PRIMARY KEY,
    order_id       INTEGER NOT NULL REFERENCES orders(order_id),
    variant_id     INTEGER NOT NULL REFERENCES product_variants(variant_id),
    product_title  VARCHAR(400) NOT NULL,    -- snapshot
    variant_title  VARCHAR(200) NOT NULL,    -- snapshot
    sku           VARCHAR(100) NOT NULL,    -- snapshot
    quantity       SMALLINT NOT NULL CHECK (quantity > 0),
    unit_price     price_cents NOT NULL,    -- price at time of order
    total_price    price_cents NOT NULL,
    UNIQUE (order_id, variant_id)
);

CREATE INDEX idx_order_items_order    ON order_items (order_id);
CREATE INDEX idx_order_items_variant ON order_items (variant_id);

-----
-- PAYMENTS
-----

CREATE TABLE payments (
    payment_id     SERIAL PRIMARY KEY,
    order_id       INTEGER NOT NULL REFERENCES orders(order_id),
    method         VARCHAR(30) NOT NULL
                  CHECK (method IN ('credit_card', 'debit_card', 'paypal',
                                     'bank_transfer', 'store_credit', 'crypto')),
    status         VARCHAR(20) NOT NULL DEFAULT 'pending'
                  CHECK (status IN
('pending', 'authorized', 'captured', 'failed', 'refunded')),
    amount_cents   price_cents NOT NULL,
    currency       VARCHAR(3) NOT NULL DEFAULT 'USD',
    gateway_ref    VARCHAR(200),
    metadata       JSONB NOT NULL DEFAULT '{}',
    processed_at   TIMESTAMPTZ,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_payments_order    ON payments (order_id);
CREATE INDEX idx_payments_status  ON payments (status);

-----
-- REVIEWS
-----

CREATE TABLE reviews (
    review_id     SERIAL PRIMARY KEY,
    product_id    INTEGER NOT NULL REFERENCES products(product_id),

```

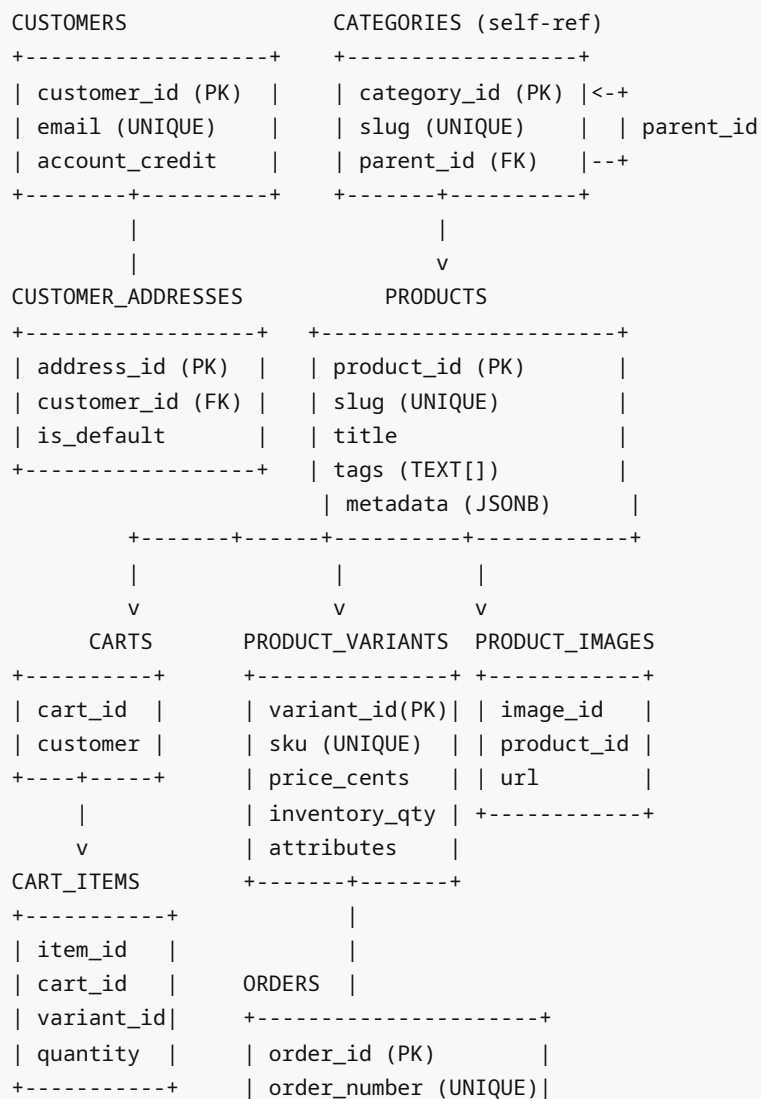
```

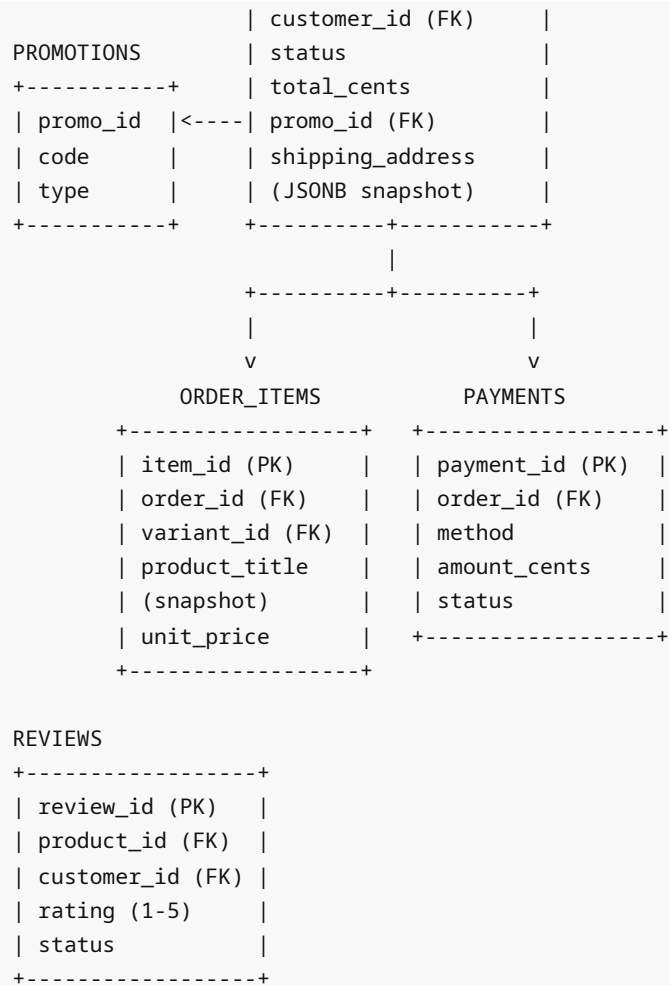
customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
order_id    INTEGER REFERENCES orders(order_id),
rating      SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
title       VARCHAR(200),
body        TEXT,
is_verified BOOLEAN NOT NULL DEFAULT FALSE,
status      VARCHAR(20) NOT NULL DEFAULT 'pending'
            CHECK (status IN ('pending','approved','rejected')),
helpful_count INTEGER NOT NULL DEFAULT 0,
created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
            UNIQUE (product_id, customer_id, order_id)
);

CREATE INDEX idx_reviews_product ON reviews (product_id, status);
CREATE INDEX idx_reviews_customer ON reviews (customer_id);

```

6. ASCII ER Diagram





7. Sample Data Scripts

```
SET search_path = shop, public;

-- Categories
INSERT INTO categories (slug, name) VALUES
    ('electronics', 'Electronics'), ('apparel', 'Apparel'), ('home', 'Home & Garden');
INSERT INTO categories (slug, name, parent_id) VALUES
    ('laptops', 'Laptops', 1), ('headphones', 'Headphones', 1), ('t-shirts', 'T-Shirts', 2);

-- Customers
INSERT INTO customers (email, first_name, last_name, is_verified) VALUES
    ('alice@example.com', 'Alice', 'Thompson', TRUE),
    ('bob@example.com', 'Bob', 'Martinez', TRUE),
    ('carol@example.com', 'Carol', 'Lee', FALSE);

-- Products
INSERT INTO products (slug, title, category_id, brand, tags, metadata)
VALUES
    ('sony-wh1000xm5', 'Sony WH-1000XM5 Headphones', 5, 'Sony',
```



```

    ARRAY['wireless','noise-cancelling','premium'],
    '{"color_options":["Black","Silver"],"connectivity":"Bluetooth 5.2"}'::JSONB),
('apple-macbook-air-m2', 'Apple MacBook Air M2', 4, 'Apple',
    ARRAY['laptop','ultrabook','apple-silicon'],
    '{"ram_gb":[8,16,24],"storage_gb":[256,512,1024]}'::JSONB),
('basic-white-tee', 'Classic White T-Shirt', 6, 'Basics Co',
    ARRAY['cotton','unisex','wardrobe-essential'],
    '{"material":"100% cotton","care":"Machine washable"}'::JSONB);

-- Product Variants
INSERT INTO product_variants (product_id, sku, title, price_cents, cost_cents,
inventory_qty, attributes)
VALUES
    (1, 'SONY-WH1000XM5-BLK', 'Black', 34999, 18000, 50, '{"color":"Black"}'::JSONB),
    (1, 'SONY-WH1000XM5-SIL', 'Silver', 34999, 18000, 30,
    '{"color":"Silver"}'::JSONB),
    (2, 'MBP-M2-8-256', '8GB / 256GB', 109900, 70000, 20,
    '{"ram":8,"storage":256}'::JSONB),
    (2, 'MBP-M2-16-512', '16GB / 512GB', 139900, 90000, 15,
    '{"ram":16,"storage":512}'::JSONB),
    (3, 'TEE-WHT-S', 'White / S', 2999, 800, 200,
    '{"color":"White","size":"S"}'::JSONB),
    (3, 'TEE-WHT-M', 'White / M', 2999, 800, 300,
    '{"color":"White","size":"M"}'::JSONB),
    (3, 'TEE-WHT-L', 'White / L', 2999, 800, 250,
    '{"color":"White","size":"L"}'::JSONB);

-- Promotions
INSERT INTO promotions (code, description, discount_type, discount_value,
min_order_cents, expires_at)
VALUES
    ('SAVE10', '10% off everything', 'percentage', 10.00, 0, NOW() +
INTERVAL '30 days'),
    ('WELCOME20', '$20 off first order', 'fixed_amount', 2000, 5000, NOW() +
INTERVAL '90 days'),
    ('FREESHIP', 'Free shipping over $50', 'free_shipping', 100.00, 5000, NOW() +
INTERVAL '60 days');

-- Orders
INSERT INTO orders (order_number, customer_id, email, status,
                    subtotal_cents, shipping_cents, tax_cents, total_cents,
                    shipping_address, promo_id)
VALUES
    ('ORD-2024-00001', 1, 'alice@example.com', 'delivered',
    34999, 0, 3150, 38149,
    '{"name":"Alice Thompson","line1":"123 Main
St","city":"Chicago","country":"US"}'::JSONB,
    NULL),
    ('ORD-2024-00002', 2, 'bob@example.com', 'shipped',
    112899, 999, 10161, 124059,
    '{"name":"Bob Martinez","line1":"456 Oak
Ave","city":"Miami","country":"US"}'::JSONB,

```

```

NULL);

INSERT INTO order_items (order_id, variant_id, product_title, variant_title, sku,
quantity, unit_price, total_price)
VALUES
(1, 1, 'Sony WH-1000XM5 Headphones', 'Black', 'SONY-WH1000XM5-BLK', 1, 34999,
34999),
(2, 3, 'Apple MacBook Air M2', '8GB/256GB', 'MBP-M2-8-256', 1, 109900,
109900),
(2, 6, 'Classic White T-Shirt', 'White / M', 'TEE-WHT-M', 3, 2999,
8997);

INSERT INTO payments (order_id, method, status, amount_cents, processed_at)
VALUES
(1, 'credit_card', 'captured', 38149, NOW() - INTERVAL '10 days'),
(2, 'paypal', 'captured', 124059, NOW() - INTERVAL '3 days');

INSERT INTO reviews (product_id, customer_id, order_id, rating, title, body,
is_verified, status)
VALUES
(1, 1, 1, 5, 'Best headphones ever', 'Incredible noise cancellation. Worth every
penny.', TRUE, 'approved');

```

8. Core SQL Queries

```

SET search_path = shop, public;

-- -----
-- Q1: Product catalog with ratings and inventory
-- -----

SELECT
  p.slug,
  p.title,
  c.name                                AS category,
  COUNT(DISTINCT pv.variant_id)         AS variant_count,
  MIN(pv.price_cents) / 100.0           AS price_from,
  MAX(pv.price_cents) / 100.0           AS price_to,
  SUM(pv.inventory_qty)                 AS total_inventory,
  ROUND(AVG(r.rating), 2)               AS avg_rating,
  COUNT(r.review_id) FILTER (WHERE r.status = 'approved') AS review_count
FROM products p
LEFT JOIN categories c      ON p.category_id = c.category_id
LEFT JOIN product_variants pv ON p.product_id = pv.product_id AND pv.is_active =
TRUE
LEFT JOIN reviews r        ON p.product_id = r.product_id AND r.status =
'approved'
WHERE p.is_active = TRUE
GROUP BY p.product_id, p.slug, p.title, c.name
ORDER BY p.is_featured DESC, p.created_at DESC;

```

```
-- -----  
-- Q2: Customer order history with totals  
-- -----
```

```
SELECT  
    o.order_number,  
    o.placed_at::DATE      AS date,  
    o.status,  
    o.total_cents / 100.0  AS total_usd,  
    COUNT(oi.item_id)     AS items,  
    p.method              AS payment_method  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN payments p     ON o.order_id = p.order_id  
WHERE o.customer_id = 1  
GROUP BY o.order_id, o.order_number, o.placed_at, o.status, o.total_cents, p.method  
ORDER BY o.placed_at DESC;
```

```
-- -----  
-- Q3: Revenue by month (current year)  
-- -----
```

```
SELECT  
    DATE_TRUNC('month', o.placed_at)::DATE      AS month,  
    COUNT(DISTINCT o.order_id)                  AS orders,  
    COUNT(DISTINCT o.customer_id)               AS customers,  
    ROUND(SUM(o.total_cents) / 100.0, 2)        AS revenue_usd,  
    ROUND(AVG(o.total_cents) / 100.0, 2)        AS avg_order_value  
FROM orders o  
JOIN payments p ON o.order_id = p.order_id  
WHERE p.status = 'captured'  
    AND o.placed_at >= DATE_TRUNC('year', NOW())  
GROUP BY 1  
ORDER BY 1;
```

```
-- -----  
-- Q4: Top-selling products by revenue  
-- -----
```

```
SELECT  
    p.title,  
    SUM(oi.quantity)                AS units_sold,  
    ROUND(SUM(oi.total_price) / 100.0, 2) AS revenue_usd,  
    ROUND(AVG(oi.unit_price) / 100.0, 2) AS avg_sell_price  
FROM order_items oi  
JOIN product_variants pv ON oi.variant_id = pv.variant_id  
JOIN products p          ON pv.product_id = p.product_id  
JOIN orders o            ON oi.order_id   = o.order_id  
WHERE o.status NOT IN ('cancelled', 'refunded')  
GROUP BY p.product_id, p.title  
ORDER BY revenue_usd DESC  
LIMIT 10;
```

```
-- -----  
-- Q5: Customer lifetime value (CLV)  
-- -----
```

```

-----
SELECT
    c.email,
    c.first_name || ' ' || c.last_name           AS customer,
    COUNT(DISTINCT o.order_id)                   AS total_orders,
    ROUND(SUM(o.total_cents) / 100.0, 2)         AS lifetime_value,
    ROUND(AVG(o.total_cents) / 100.0, 2)         AS avg_order_value,
    MIN(o.placed_at)::DATE                       AS first_order,
    MAX(o.placed_at)::DATE                       AS last_order,
    CURRENT_DATE - MAX(o.placed_at)::DATE        AS days_since_last_order
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status NOT IN ('cancelled')
GROUP BY c.customer_id, c.email, c.first_name, c.last_name
ORDER BY lifetime_value DESC;

```

```

-----
-- Q6: Cart abandonment rate
-----

```

```

WITH cart_status AS (
    SELECT
        c.cart_id,
        c.customer_id,
        c.created_at,
        COUNT(ci.item_id) AS item_count,
        SUM(ci.quantity * pv.price_cents) / 100.0 AS cart_value,
        EXISTS (
            SELECT 1 FROM orders o
            WHERE o.customer_id = c.customer_id
            AND o.placed_at > c.created_at
            AND o.placed_at < c.expires_at
        ) AS converted
    FROM carts c
    JOIN cart_items ci      ON c.cart_id = ci.cart_id
    JOIN product_variants pv ON ci.variant_id = pv.variant_id
    GROUP BY c.cart_id, c.customer_id, c.created_at, c.expires_at
)
SELECT
    COUNT(*)                               AS total_carts_with_items,
    COUNT(*) FILTER (WHERE converted)       AS converted,
    COUNT(*) FILTER (WHERE NOT converted)    AS abandoned,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT converted) / COUNT(*), 1) AS
abandonment_rate_pct,
    ROUND(AVG(cart_value) FILTER (WHERE NOT converted), 2) AS avg_abandoned_value
FROM cart_status;

```

```

-----
-- Q7: Variant inventory status
-----

```

```

SELECT
    p.title,
    pv.title           AS variant,

```

```

    pv.sku,
    pv.price_cents / 100.0 AS price,
    pv.inventory_qty,
    CASE
        WHEN pv.inventory_qty = 0 THEN 'Out of Stock'
        WHEN pv.inventory_qty < 10 THEN 'Low Stock'
        WHEN pv.inventory_qty < 50 THEN 'In Stock'
        ELSE 'Well Stocked'
    END AS stock_status,
    pv.attributes->>'color' AS color,
    pv.attributes->>'size' AS size
FROM product_variants pv
JOIN products p ON pv.product_id = p.product_id
WHERE pv.is_active = TRUE
ORDER BY p.title, pv.title;

-- -----
-- Q8: Review summary with star distribution
-- -----

SELECT
    p.title,
    COUNT(r.review_id) AS total_reviews,
    ROUND(AVG(r.rating), 2) AS avg_rating,
    COUNT(*) FILTER (WHERE r.rating = 5) AS five_star,
    COUNT(*) FILTER (WHERE r.rating = 4) AS four_star,
    COUNT(*) FILTER (WHERE r.rating = 3) AS three_star,
    COUNT(*) FILTER (WHERE r.rating <= 2) AS low_rating
FROM products p
LEFT JOIN reviews r ON p.product_id = r.product_id AND r.status = 'approved'
GROUP BY p.product_id, p.title
ORDER BY avg_rating DESC NULLS LAST;

-- -----
-- Q9: Order status funnel
-- -----

SELECT
    status,
    COUNT(*) AS count,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct
FROM orders
GROUP BY status
ORDER BY COUNT(*) DESC;

-- -----
-- Q10: LATERAL join - top 2 products per category
-- -----

SELECT
    c.name AS category,
    top.title,
    top.revenue_usd
FROM categories c
CROSS JOIN LATERAL (

```

```

SELECT
    p.title,
    ROUND(SUM(oi.total_price) / 100.0, 2) AS revenue_usd
FROM products p
JOIN product_variants pv ON p.product_id = pv.product_id
JOIN order_items oi      ON pv.variant_id = oi.variant_id
JOIN orders o            ON oi.order_id   = o.order_id
WHERE p.category_id = c.category_id
    AND o.status NOT IN ('cancelled', 'refunded')
GROUP BY p.product_id, p.title
ORDER BY revenue_usd DESC
LIMIT 2
) top
ORDER BY c.name, top.revenue_usd DESC;

-- -----
-- Q11: Promotion effectiveness
-- -----

SELECT
    pr.code,
    pr.discount_type,
    pr.discount_value,
    COUNT(o.order_id) AS orders_used,
    ROUND(AVG(o.total_cents) / 100.0, 2) AS avg_order_total,
    ROUND(SUM(o.discount_cents) / 100.0, 2) AS total_discount_given,
    ROUND(SUM(o.total_cents) / 100.0, 2) AS total_revenue
FROM promotions pr
LEFT JOIN orders o ON pr.promo_id = o.promo_id
    AND o.status NOT IN ('cancelled')
GROUP BY pr.promo_id, pr.code, pr.discount_type, pr.discount_value
ORDER BY orders_used DESC;

-- -----
-- Q12: Cohort retention (by signup month)
-- -----

WITH first_orders AS (
    SELECT customer_id, DATE_TRUNC('month', MIN(placed_at)) AS cohort_month
    FROM orders
    WHERE status NOT IN ('cancelled')
    GROUP BY customer_id
),
monthly_orders AS (
    SELECT o.customer_id, DATE_TRUNC('month', o.placed_at) AS order_month
    FROM orders o WHERE status NOT IN ('cancelled')
)
SELECT
    fo.cohort_month::DATE,
    COUNT(DISTINCT fo.customer_id) AS cohort_size,
    COUNT(DISTINCT mo.customer_id)
    FILTER (WHERE mo.order_month = fo.cohort_month + INTERVAL '1 month') AS
m1_retention,
    COUNT(DISTINCT mo.customer_id)

```

```

        FILTER (WHERE mo.order_month = fo.cohort_month + INTERVAL '2 months') AS
m2_retention
FROM first_orders fo
LEFT JOIN monthly_orders mo ON fo.customer_id = mo.customer_id
GROUP BY fo.cohort_month
ORDER BY fo.cohort_month;

-- -----
-- Q13: Products with JSONB attribute filtering
-- -----

SELECT
    p.title,
    pv.attributes->>'color' AS color,
    pv.attributes->>'size' AS size,
    pv.price_cents / 100.0 AS price,
    pv.inventory_qty
FROM product_variants pv
JOIN products p ON pv.product_id = p.product_id
WHERE pv.attributes @> '{"color":"White"}'::JSONB
    AND pv.inventory_qty > 0
ORDER BY pv.price_cents;

```

9. Stored Procedures / Functions

```

-- -----
-- FUNCTION 1: Place an order from cart
-- -----

CREATE OR REPLACE FUNCTION shop.place_order(
    p_cart_id    UUID,
    p_email      VARCHAR,
    p_ship_addr  JSONB,
    p_bill_addr  JSONB DEFAULT NULL
)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_cart        shop.carts%ROWTYPE;
    v_item        RECORD;
    v_variant     shop.product_variants%ROWTYPE;
    v_subtotal    INTEGER := 0;
    v_order_id    INTEGER;
    v_order_num   VARCHAR;
    v_promo       shop.promotions%ROWTYPE;
    v_discount    INTEGER := 0;
BEGIN
    SELECT * INTO v_cart FROM shop.carts WHERE cart_id = p_cart_id;
    IF NOT FOUND THEN RAISE EXCEPTION 'Cart % not found', p_cart_id; END IF;

    -- Create order number
    v_order_num := 'ORD-' || TO_CHAR(NOW(), 'YYYY') || '-' ||

```

```

        LPAD(nextval('shop.orders_order_id_seq')::TEXT, 5, '0');

-- Calculate subtotal
FOR v_item IN
    SELECT ci.quantity, pv.price_cents, pv.variant_id, pv.inventory_qty,
           pv.inventory_policy, p.title AS product_title, pv.title AS
variant_title, pv.sku
    FROM shop.cart_items ci
    JOIN shop.product_variants pv ON ci.variant_id = pv.variant_id
    JOIN shop.products p          ON pv.product_id = p.product_id
    WHERE ci.cart_id = p_cart_id
    FOR UPDATE OF pv
LOOP
    IF v_item.inventory_qty < v_item.quantity AND v_item.inventory_policy =
'deny' THEN
        RAISE EXCEPTION 'Insufficient stock for SKU %', v_item.sku;
    END IF;
    v_subtotal := v_subtotal + (v_item.price_cents * v_item.quantity);
END LOOP;

-- Apply promo
IF v_cart.promo_code IS NOT NULL THEN
    SELECT * INTO v_promo FROM shop.promotions WHERE code = v_cart.promo_code
AND is_active = TRUE;
    IF FOUND AND v_subtotal >= v_promo.min_order_cents THEN
        v_discount := CASE v_promo.discount_type
            WHEN 'percentage' THEN (v_subtotal * v_promo.discount_value /
100)::INTEGER
            WHEN 'fixed_amount' THEN LEAST(v_promo.discount_value::INTEGER *
100, v_subtotal)
            ELSE 0
        END;
    END IF;
END IF;

INSERT INTO shop.orders (order_number, customer_id, email, subtotal_cents,
discount_cents,
                        tax_cents, total_cents, shipping_address,
billing_address, promo_id)
VALUES (
    v_order_num,
    v_cart.customer_id,
    p_email,
    v_subtotal,
    v_discount,
    ((v_subtotal - v_discount) * 0.09)::INTEGER,
    (v_subtotal - v_discount + ((v_subtotal - v_discount) * 0.09)::INTEGER),
    p_ship_addr,
    COALESCE(p_bill_addr, p_ship_addr),
    v_promo.promo_id
)
RETURNING order_id INTO v_order_id;

```



```

-- Insert order items and decrement inventory
FOR v_item IN
    SELECT ci.quantity, pv.price_cents, pv.variant_id, p.title AS product_title,
           pv.title AS variant_title, pv.sku
    FROM shop.cart_items ci
    JOIN shop.product_variants pv ON ci.variant_id = pv.variant_id
    JOIN shop.products p ON pv.product_id = p.product_id
    WHERE ci.cart_id = p_cart_id
LOOP
    INSERT INTO shop.order_items (order_id, variant_id, product_title,
variant_title, sku,
                                quantity, unit_price, total_price)
    VALUES (v_order_id, v_item.variant_id, v_item.product_title,
v_item.variant_title,
            v_item.sku, v_item.quantity, v_item.price_cents,
            v_item.price_cents * v_item.quantity);

    UPDATE shop.product_variants
    SET inventory_qty = GREATEST(0, inventory_qty - v_item.quantity)
    WHERE variant_id = v_item.variant_id;
END LOOP;

-- Clear cart
DELETE FROM shop.cart_items WHERE cart_id = p_cart_id;

RETURN v_order_id;
END;
$$;

-----
-- FUNCTION 2: Process a refund
-----

CREATE OR REPLACE FUNCTION shop.process_refund(
    p_order_id    INTEGER,
    p_amount_cents INTEGER,
    p_reason       TEXT DEFAULT NULL
)
RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    v_order    shop.orders%ROWTYPE;
    v_paid      INTEGER;
BEGIN
    SELECT * INTO v_order FROM shop.orders WHERE order_id = p_order_id FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'Order % not found', p_order_id; END IF;
    IF v_order.status = 'refunded' THEN
        RAISE EXCEPTION 'Order % is already fully refunded', p_order_id;
    END IF;

    SELECT COALESCE(SUM(amount_cents),0) INTO v_paid
    FROM shop.payments WHERE order_id = p_order_id AND status = 'captured';

```

```

IF p_amount_cents > v_paid THEN
    RAISE EXCEPTION 'Refund amount $$ exceeds captured payments $$',
        p_amount_cents/100.0, v_paid/100.0;
END IF;

INSERT INTO shop.payments (order_id, method, status, amount_cents, metadata,
    processed_at)
VALUES (p_order_id, 'credit_card', 'refunded', p_amount_cents,
    jsonb_build_object('reason', p_reason), NOW());

UPDATE shop.orders
SET status = CASE WHEN p_amount_cents = total_cents THEN 'refunded' ELSE
    'processing' END,
    updated_at = NOW()
WHERE order_id = p_order_id;

RETURN format('Refund of $$s processed for order %s', p_amount_cents/100.0,
    p_order_id);
END;
$$;

```

10. Triggers

```

-- TRIGGER 1: Enforce valid order status transitions
CREATE OR REPLACE FUNCTION shop.trg_order_status_transition()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.status = NEW.status THEN RETURN NEW; END IF;
    IF NOT (
        (OLD.status = 'pending' AND NEW.status IN ('confirmed', 'cancelled')) OR
        (OLD.status = 'confirmed' AND NEW.status IN ('processing', 'cancelled')) OR
        (OLD.status = 'processing' AND NEW.status IN ('shipped', 'cancelled')) OR
        (OLD.status = 'shipped' AND NEW.status IN ('delivered', 'refunded')) OR
        (OLD.status = 'delivered' AND NEW.status IN ('refunded'))
    ) THEN
        RAISE EXCEPTION 'Invalid order status transition: % -> %', OLD.status,
            NEW.status;
    END IF;
    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_order_transitions
BEFORE UPDATE OF status ON shop.orders
FOR EACH ROW EXECUTE FUNCTION shop.trg_order_status_transition();

-- TRIGGER 2: Bump version on order updates (optimistic locking)
CREATE OR REPLACE FUNCTION shop.trg_bump_version()

```

```
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.version := OLD.version + 1;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_orders_version
BEFORE UPDATE ON shop.orders
FOR EACH ROW EXECUTE FUNCTION shop.trg_bump_version();
```

11. Performance Optimization

```
-- Partial indexes for active state
CREATE INDEX idx_variants_low_stock ON product_variants (product_id)
    WHERE inventory_qty < 10 AND is_active = TRUE;

-- GIN index on JSONB attributes for faceted filtering
CREATE INDEX idx_variants_attrs_gin ON product_variants USING GIN (attributes);

-- Covering index for order lookup
CREATE INDEX idx_orders_covering ON orders (customer_id, placed_at DESC)
    INCLUDE (order_number, status, total_cents);
```

12. Extensions Used

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;    -- gen_random_uuid() for cart IDs
CREATE EXTENSION IF NOT EXISTS pg_trgm;     -- fuzzy product search
CREATE EXTENSION IF NOT EXISTS btree_gin;    -- combine GIN + btree operations
```

13. Testing Guide

```
-- TEST 1: Add to cart and place order
INSERT INTO shop.carts (customer_id) VALUES (3) RETURNING cart_id;
-- Use returned cart_id in next step
INSERT INTO shop.cart_items (cart_id, variant_id, quantity) VALUES ('<cart_id>', 5, 2);
SELECT shop.place_order('<cart_id>', 'carol@example.com', '{"name":"Carol Lee","line1":"789 Pine St","city":"Boston","country":"US"}'::JSONB);

-- TEST 2: Invalid status transition
UPDATE shop.orders SET status = 'delivered' WHERE order_id = 1; -- pending->delivered should fail

-- TEST 3: Refund
SELECT shop.process_refund(1, 38149, 'Customer not satisfied');
```

```
-- TEST 4: JSONB attribute filtering (Q13)
-- Verify WHERE attributes @> '{"color":"White"}' returns correct variants

-- TEST 5: Revenue report (Q3) - verify totals match inserted order data
```

14. Extension Challenges

- 1. Search & Faceted Filtering:** Build a full-text + JSONB attribute search function. Accept filters like `color=White AND price_max=5000`. Return ranked results using `ts_rank` combined with attribute scores.
- 2. Wishlist and Saved Items:** Add a `wishlists` table (customer can have multiple named wishlists). Track when items are moved from wishlist to cart. Report on most-wishlisted items that have not been purchased.
- 3. Subscription Orders:** Add a `subscriptions` table for recurring orders (weekly/monthly). Write a PL/pgSQL job that generates new orders from active subscriptions and charges stored payment methods.
- 4. Dynamic Pricing:** Add a `price_rules` table that overrides variant prices based on time of day, customer segment, or inventory level. Write a function `get_effective_price(variant_id, customer_id)` that applies the best rule.
- 5. Seller Marketplace:** Extend the schema to support multiple sellers. Add a `sellers` table, link products to sellers, and split order items by seller. Calculate seller payouts after platform commission.

15. What You Learned

Skill	Demonstrated By
Prices in cents (integer)	Avoid floating-point issues
JSONB for flexible attributes	Product metadata, variant attributes, address snapshots
State machine enforcement	Order status transition trigger
Optimistic locking	<code>version</code> column bump trigger
LATERAL joins	Top-N products per category
Cohort analysis	Customer retention window function
UUID primary keys	Cart table uses <code>gen_random_uuid()</code>
Array columns	Product tags as <code>TEXT[]</code>
GIN indexes	Full-text + JSONB + array filtering
Multi-step transaction	<code>place_order</code> function spans inventory + order + items